

PROGRAMAÇÃO ORIENTADA A OBJETOS

Aula 11 – Herança

Análise e Desenvolvimento de Sistemas

Prof.: Cícero Santos

Agenda de Aula

- **Herança P00**

- Entender o que é herança
- Aprender noções de superclasse e subclasse
- Extends: para herdar características e comportamentos
- Acesso a membros da superclasse

Herança

- **Conceito**

- De acordo com Thiago Leite e Carvalho, a herança é uma ferramenta de especialização. Imagine que temos um contexto geral e precisamos modelar necessidades específicas a partir dele.
- Deitel (2010) complementa essa ideia utilizando uma analogia com o mundo automobilístico: um objeto da classe "Conversível" é, com certeza, um objeto da classe mais geral "Automóvel", mas com a característica específica de que o teto pode ser levantado ou baixado.

Herança

- **Conceito**

- A herança será aplicada tanto para características quanto para comportamento.
- Permitindo basear uma nova classe a partir de outra classe.

Terminologias importantes



- **Superclasse**
 - A classe existente “mais genérica” (Pessoa ou Automovel)
- **Subclasse**
 - A nova classe “mais especifica”, quem herda os membros de superclasse (Aluno, Fusca)
- **Relacionamento**
 - “É um”, a herança representa um objeto da subclasse na superclasse.

Analogia Herança Pessoa



Analogia Herança Pessoa

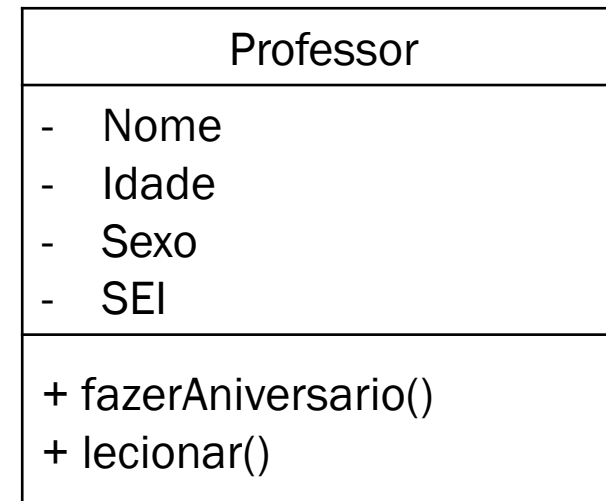


Analogia Herança Pessoa



Medico
- Nome - Idade - Sexo - CRM
+ fazerAniversario() + atenderPaciente()

Analogia Herança Pessoa



Analogia Herança Pessoa



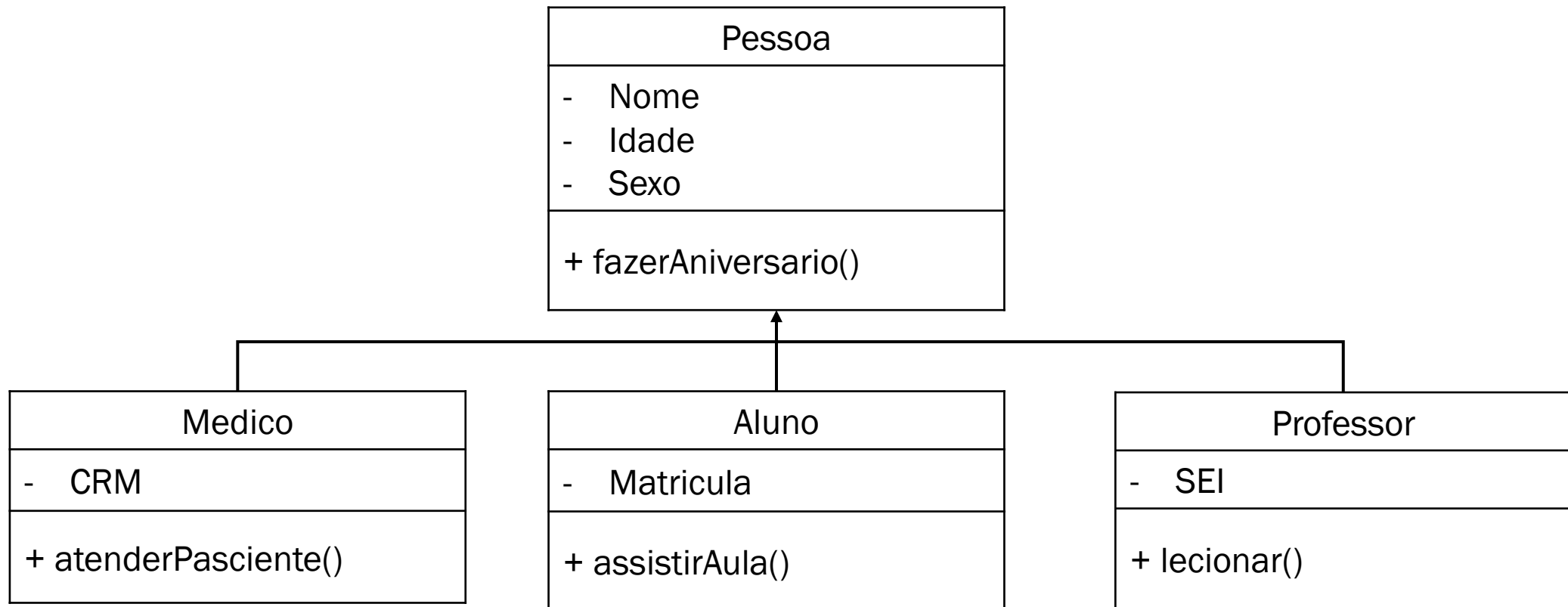
Aluno
- Nome - Idade - Sexo - Matricula
+ fazerAniversario() + assistirAula()

Analogia Herança Pessoa



Pessoa
- Nome - Idade - Sexo
+ fazerAniversario()

Herança Representação UML



Pontos Importantes

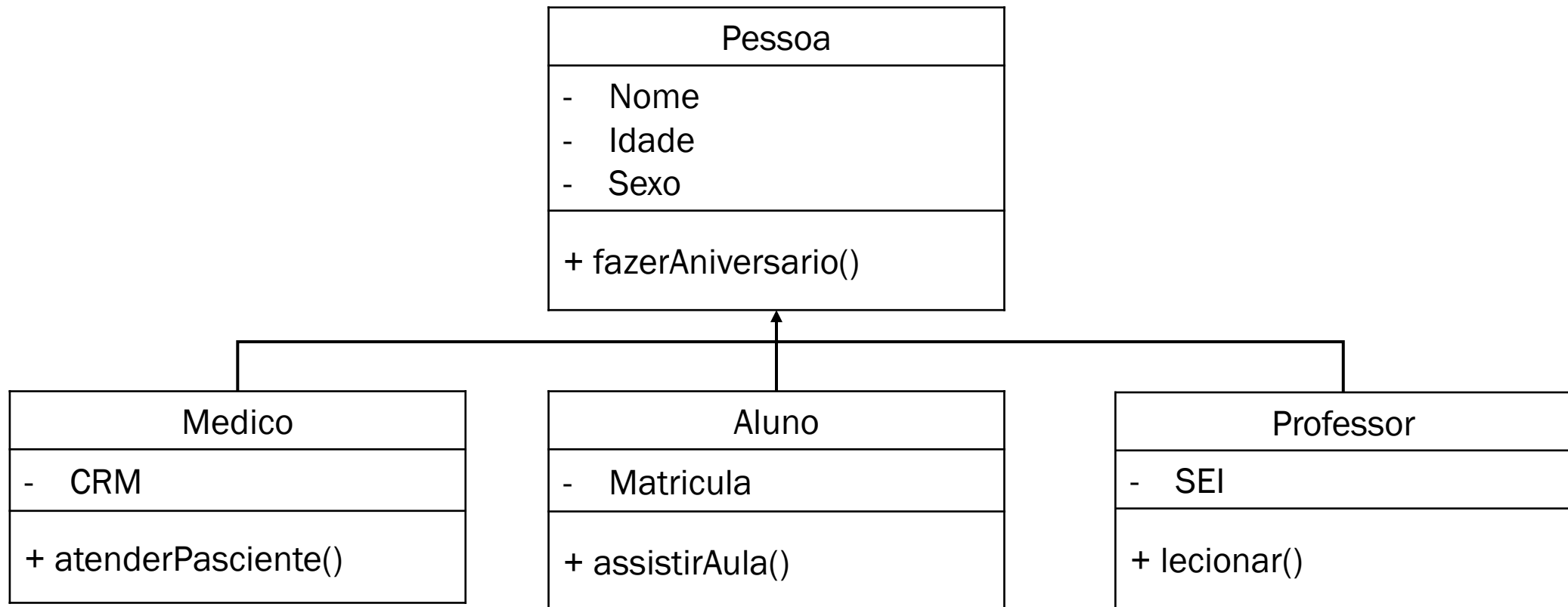
- **Extends**
 - Indica que FuncionarioComissionado herda de Funcionario.
- **super()**
 - O construtor da subclasse deve chamar o construtor da superclasse para inicializar os membros herdados.
- **Encapsulamento**
 - Membros private da superclasse não são acessíveis diretamente pela subclasse. Para permitir esse acesso mantendo a segurança, Carvalho sugere o uso de métodos get/set ou o modificador protected.

Na Prática

- Exemplo do uso da herança.

```
public class Pessoa {  
    private String nome;  
  
}  
  
public class Aluno extends Pessoa {  
    // Segue o jogo  
  
}
```

Herança Representação UML



Desafio

- **Crie uma classe com os dados apresentados no slide anterior**
 - No main, criar:
 - P1 = new Pessoa()
 - P2 = new Medico()
 - P3 = new Professor()
 - P4 = new Aluno()
 - Setar e mostrar os dados de todas as classes especialistas.

Sobrecarga de Métodos

- **Sobrescrita de Métodos (@Override)**
 - Muitas vezes, a subclasse precisa de um comportamento diferente do que foi definido na superclasse.
 - A subclasse pode acrescentar ou eliminar algo do comportamento herdado.

Sobrecarga de Métodos

- **Exemplo**

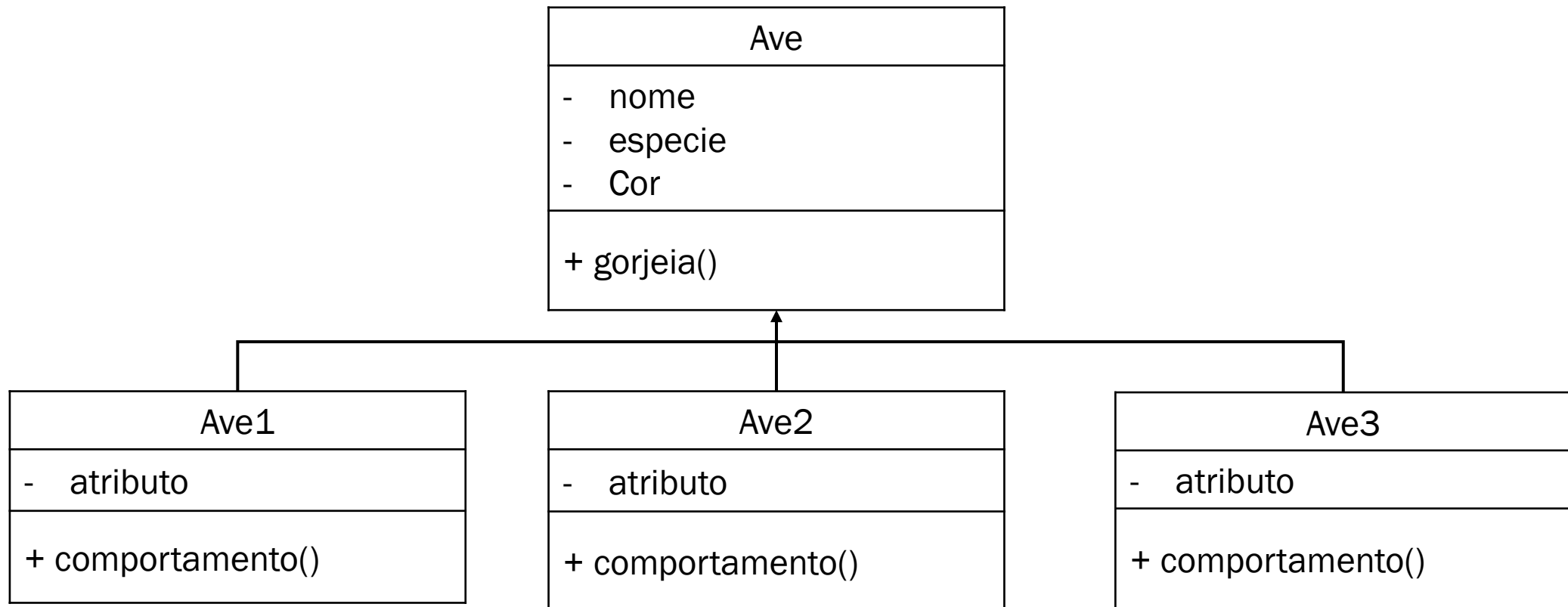
Se a classe **Pessoa** tem um método `fazerAniversario()`, a subclasse **Aluno** deve sobrescrever esse método para calcular seguindo a regra do negócio.

```
@Override  
public void fazerAniversario() {  
    this.idade + 2;  
}
```

Sobrecarga de Métodos

- A anotação **@Override** é uma boa prática que indica ao compilador que estamos redefinindo um método da superclasse.

Herança Representação UML



Continuando....



- **O que já vimos**
 - Entender o que é herança
 - Aprender noções de superclasse e subclasse
 - Extends: para herdar características e comportamentos
 - Acesso a membros da superclasse

Agenda de Aula

- **Herança P00**
 - Generalização
 - Especialização
 - Classe abstrata
 - Herança de implementação (pobre)
 - Herança para diferença
 - Classe final
 - Método final
 - Métodos abstratos

Generalização

- **Generalização: Do Específico para o Comum**

- A Generalização é o processo de identificar características e comportamentos comuns em várias classes e "subi-los" para uma classe mais genérica (a superclasse).
 - É uma forma de abstração. Se você percebe que **Carro**, **Moto** e **Caminhão** possuem todos um **chassi** e o método **ligar()**, você extrai esses pontos comuns para uma superclasse **Veículo**.
 - Reduzir a redundância e facilitar a manutenção. Se o comportamento de "ligar" mudar para todos os veículos, você altera em apenas um lugar.

Especialização

- **Especialização: Do Comum para o Específico**

- A Especialização é o inverso. É quando você cria uma nova classe a partir de uma existente para adicionar comportamentos ou estados que a classe original não possuía.
 - A subclasse é uma versão mais específica da sua superclasse. Ela herda tudo o que a superclasse tem, mas se torna "especialista" em algo.
 - A classe `Funcionario` é geral. A classe `Diretor` é uma especialização, pois além de ser um funcionário, ela possui uma `gratificacaoDeCargo` e métodos de `tomadaDeDecisao`.

Classe abstract/abstrata

- **Conceito**

- São classes que às vezes projetamos, porém não pretendemos criar objetos. Essas classes são chamadas de classes abstratas.
- Elas são úteis para representar conceitos genéricos em uma hierarquia de herança.

~~Pessoa p = new Pessoa();~~

```
public abstract class Pessoa {  
    private String nome;  
    private String sexo;  
    private int idade;  
}  
  
public class Aluno extends Pessoa {  
    // procedimentos  
}
```

Herança de implementação (herança pobre)

- **Conceito**

- A herança de implementação ocorre quando uma subclasse estende uma superclasse puramente para reutilizar seus métodos e atributos, ignorando se a subclasse realmente "é uma" versão especializada da superclasse.
- No exemplo ao lado: A classe **Professor** vai herdar tudo de **Pessoa**, porém nada será implementado.

```
public class Professor extends Pessoa {}  
  
Professor p = new Professor()  
p.setNome("Cícero");
```

Herança para diferença

- **Conceito**

- Além de herdar todas as características e comportamento, na subclasse eu tenho uma especialização.
- No exemplo ao lado: A classe **Estagiario** vai herdar tudo de **Pessoa**, porém o método de **pagarMensalidade** tem uma característica específica nesta subclasse.

```
public class Estagiario extends Pessoa {  
  
    @Override  
    public void pagarMensalidade(){  
        this.mensalidade = 0;  
    }  
}  
  
Estagiario e = new Estagiario()  
e. pagarMensalidade();
```

Método: final

- **Conceito**

- Um método declarado como final não pode ser sobrescrito (*overridden*) em nenhum
- Você utiliza **final** quando deseja garantir que a implementação de um método seja preservada por toda a hierarquia, impedindo que subclasses alterem seu comportamento principal.

```
public final void fazerEmprestimo {  
    // to-do  
}  
  
@Override  
public void fazerEmprestimo(){  
    this.parcelas = 0;  
}
```

Classe: final

- **Conceito**

- Uma classe **final** é aquela que não pode ser estendida. Ou seja, ela não pode ter subclasses.
- Segurança e imutabilidade. Se uma classe é final, todos os seus métodos são implicitamente final.
- No exemplo ao lado a classe **final** **Pessoa**, não nos permite herdar para uma subclasse **Estagiario**.

```
// classe final
public final class Pessoa {}

// não é permitido herança
public class Estagiario extends Pessoa {}
```

Construtor: super

- **Classe pai (superclasse)**
 - Quando trabalhamos com construtores em uma classe pai e herdamos em uma classe filha devemos invocar para garantirmos a inicialização corretas dos dados.

```
public class Pessoa {  
    private String nome;  
  
    public Pessoa(String nome){  
        this.nome = nome;  
    }  
}  
  
public class Estagiario extends Pessoa {  
    public Estagiario(String){  
        super(nome);  
    }  
}
```

Bora praticar...

Se liga nas instruções do professor.

Referência



DEITEL, Paul; DEITEL, Harvey. Java: como programar. 10. ed. São Paulo: Pearson Education do Brasil, 2016.

CARVALHO, Thiago Leite. Orientação a Objetos: aprenda seus conceitos e suas aplicabilidades de forma efetiva. 1. ed. São Paulo: Casa do Código, 2016.

Obrigado.

Aula 11 – Herança

Análise e Desenvolvimento de Sistemas

Prof.: Cícero Santos

